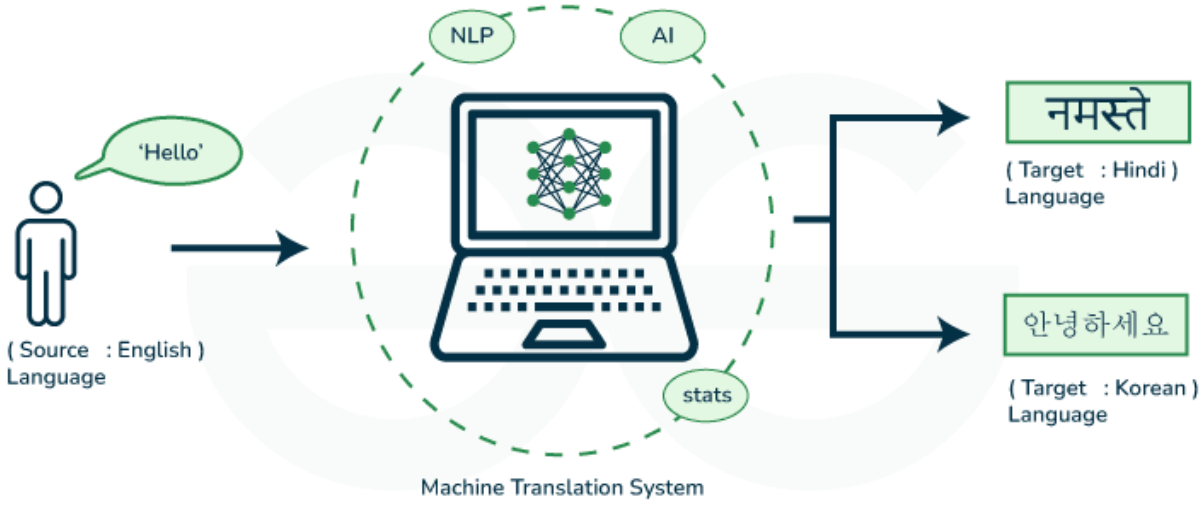
Basics for GenAI

Transformers

Jimmy Shong

Transformers Have Revolutionized the Field of ML

- Originally developed to solve machine translation
- Underpins virtually every state-of-the-art NLP and computer vision model
- Led to rise of large foundational models in the industry



Microsoft + OpenAI

Google + DeepMind

TESLA + xAI

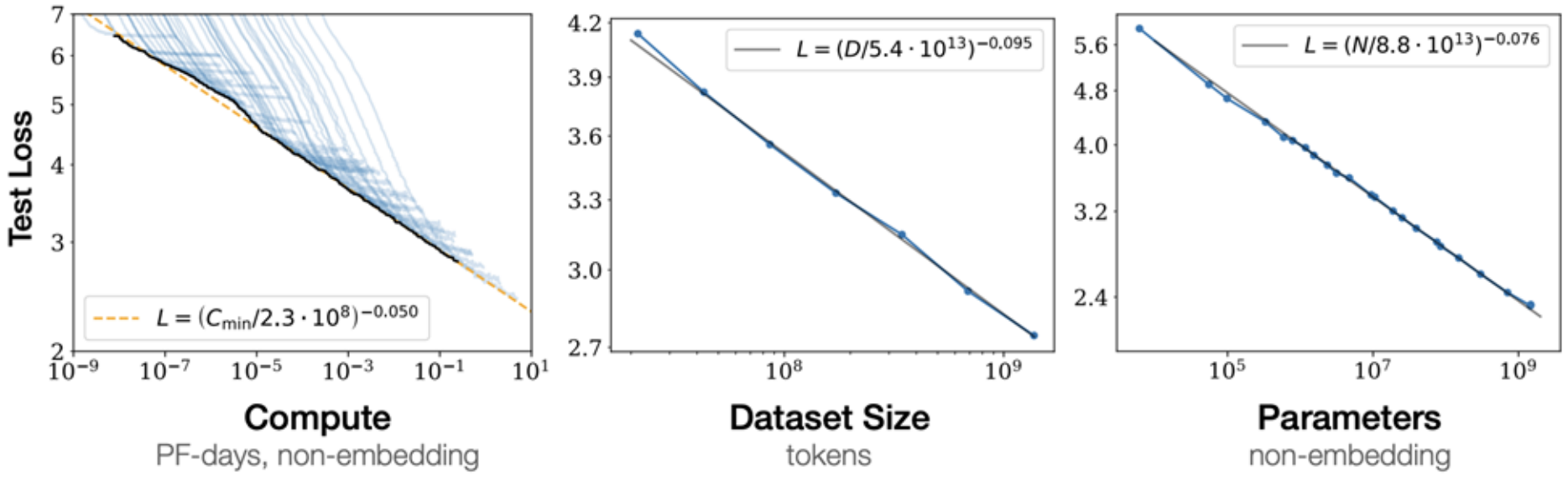
Meta + Meta AI

amazon + ANTHROPIC

www.therundown.ai

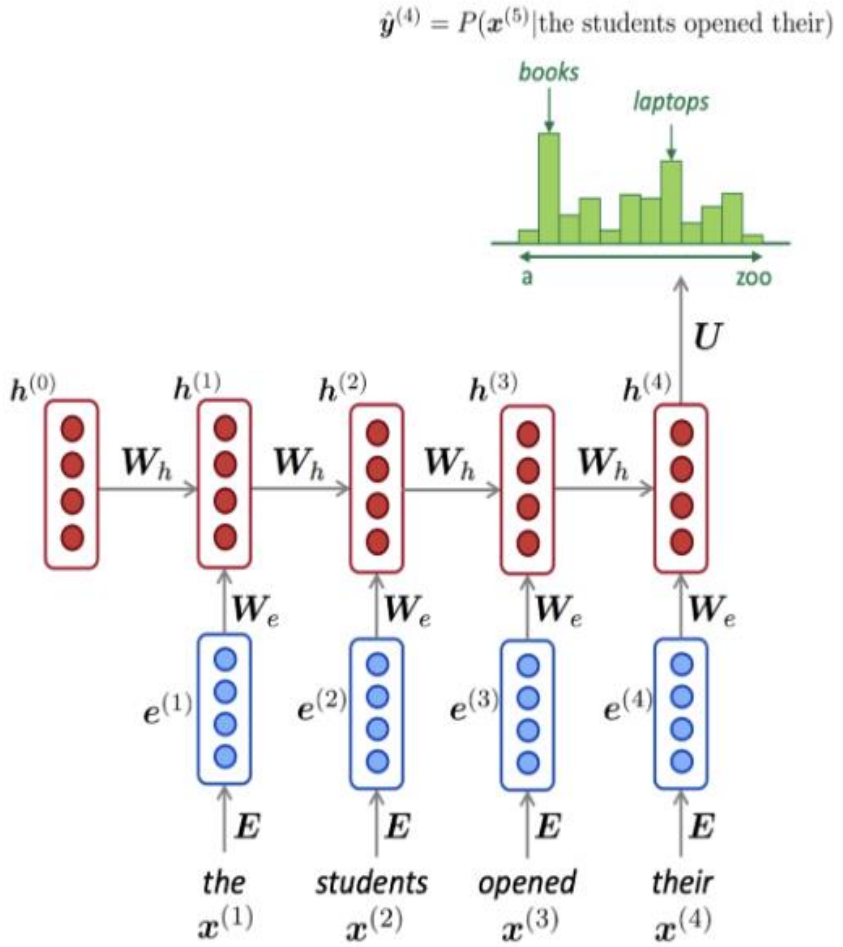
Transformer Scaling Laws

- With Transformers, language modeling performance improves smoothly as we increase model size, training data, and compute resources in tandem.
- This power-law relationship has been observed over multiple orders of magnitude with no concrete sign of slowing.



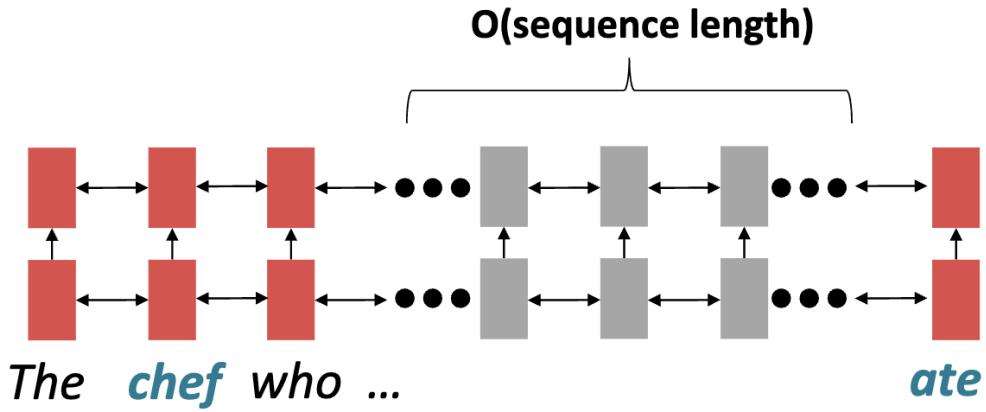
Prior to Transformers

- The de facto strategy in NLP was to encode sentences with an RNN
- Define your output (parse, sentence, summary) as a sequence and use an RNN to generate it.



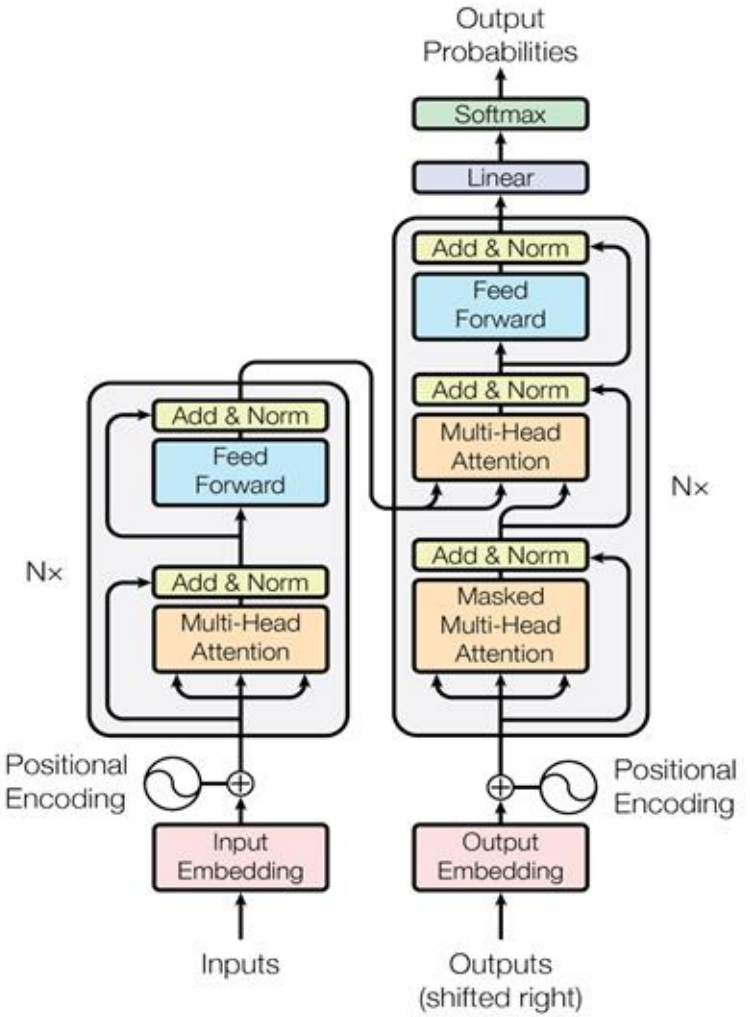
Problems of RNNs / Motivation for Transformers

- Sequential computation prevents parallelization
- Difficult to access information from many steps back due vanishing/ exploding gradients
- We want to do the following:
  - Maximize the amount of computation that can be parallelized.
  - Do not increase computational complexity per layer



Original Transformer Architecture

- Proposed transformer model, entirely built on self-attention mechanism without using sequence-aligned recurrent architectures
- Model Architecture
  - Sinusoidal Positional Encoding
  - Encoder
    - Multi-Head Self-Attention
    - Feed-Forward (MLP) Layer with ReLU
    - Layer Normalization
  - Decoder
    - Masked Multi-Head Self-Attention
    - Multi-Head Cross-Attention
    - Feed-Forward (MLP) Layer with ReLU
    - Usually LLMs are Decoder-only

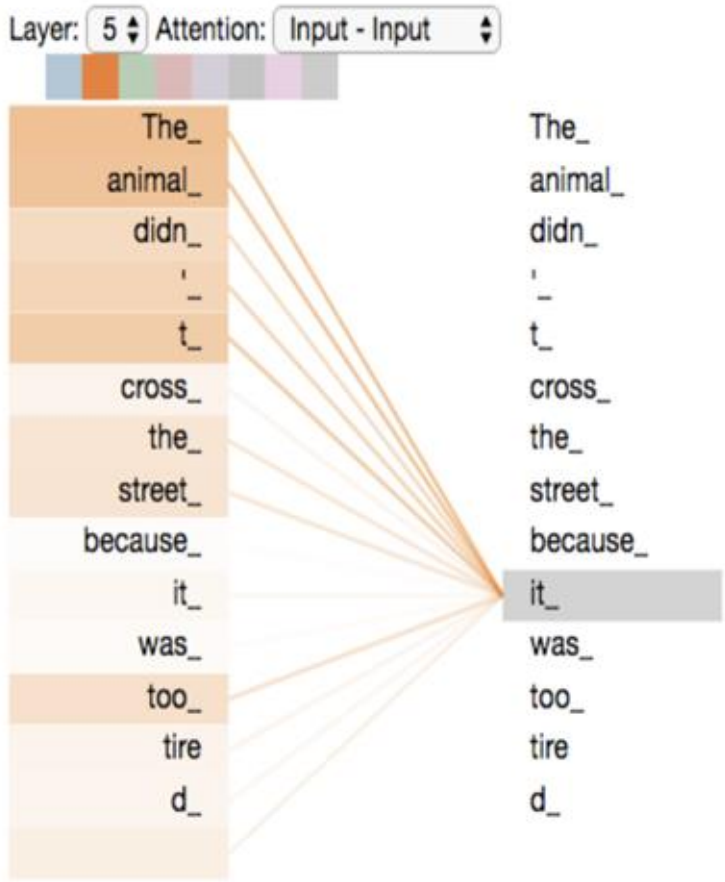


Self-Attention at High Level

- Consider the following example:
  - The animal didn't cross the street because **it** was too tired
  - The animal didn't cross the street because **it** was too wide

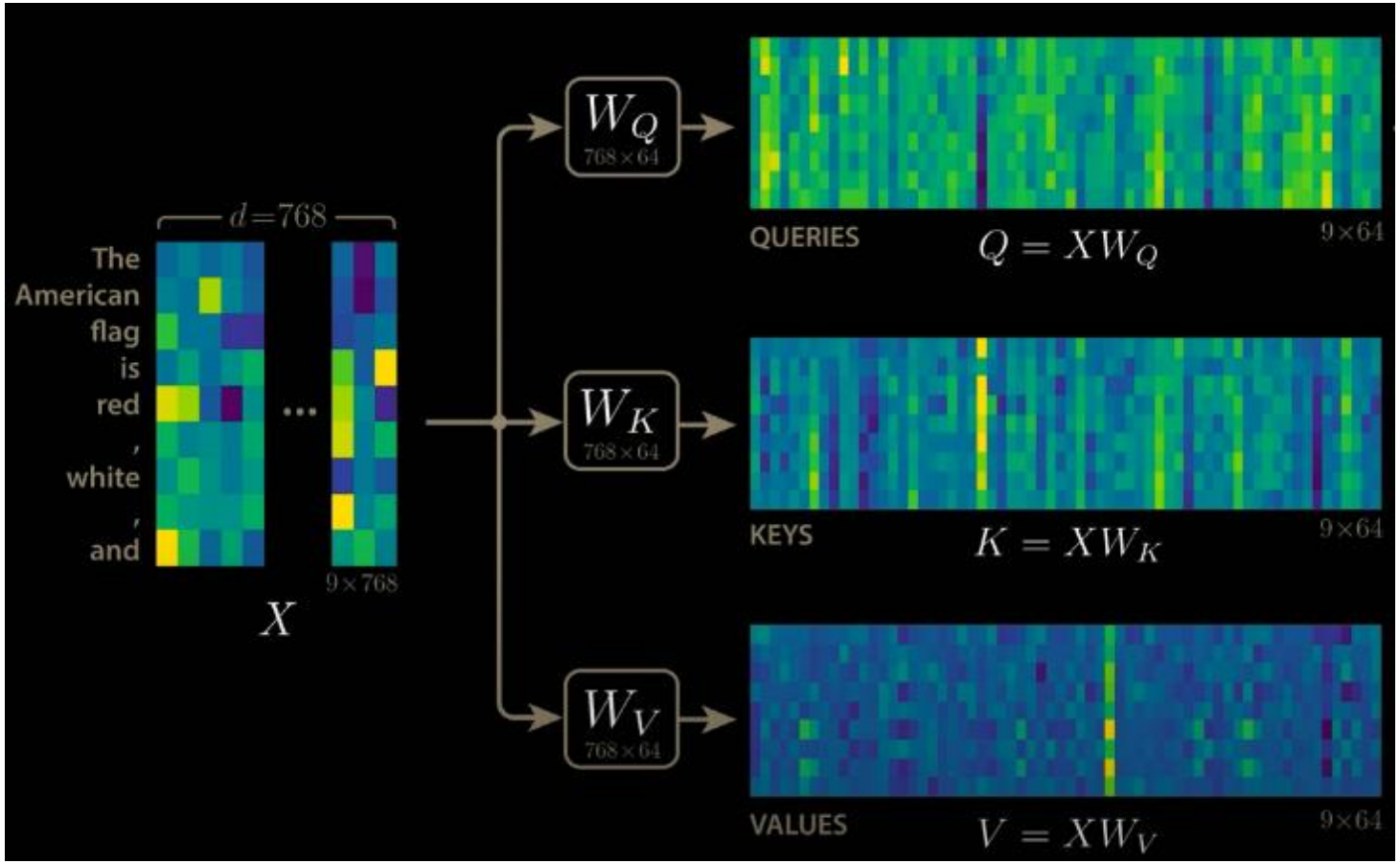
- As the model processes each word, self-attention allows the model to look at other positions in input sequence to help get a better encoding

- This capability is handled by a set of learned weights that determine how much attention should be paid to each input at each time step



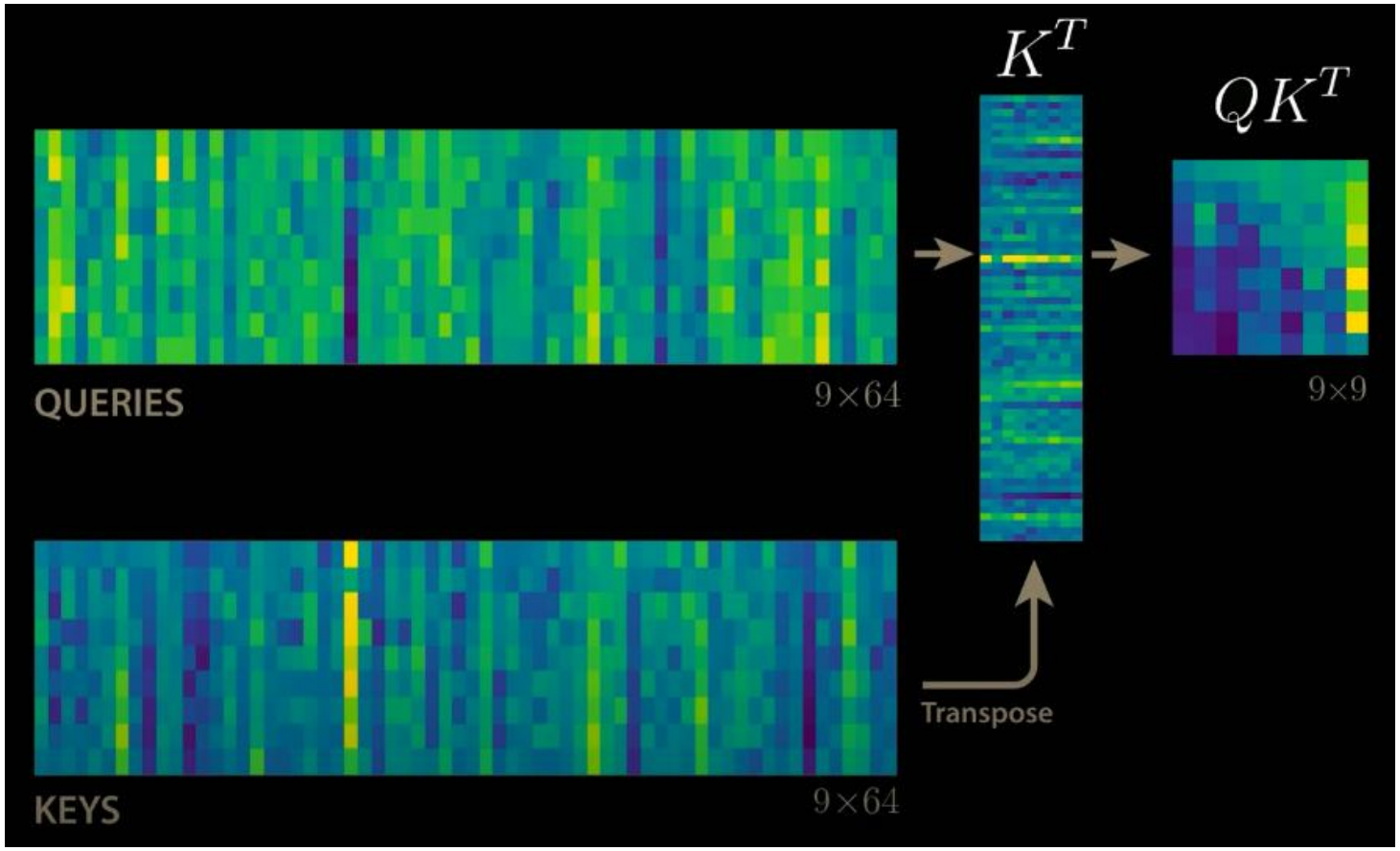
Self-Attention

- Step 1: Create three matrix from input matrix (X)
  - Query Vector (Q)
  - Key Vector (K)
  - Value Vector (V)
- These are created by multiplying x_i with learnable matrices W^Q,W^K, W^V



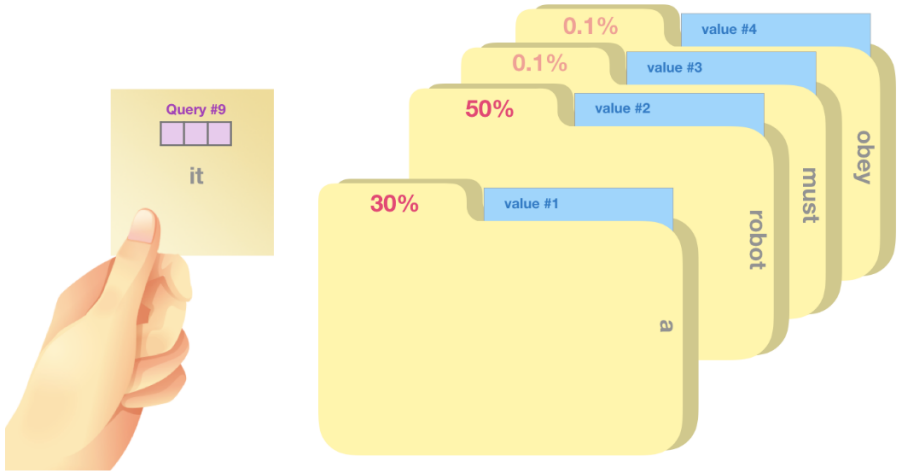
Dot Product Attention

- Step 2: Calculate self-attention scores - score all words of input sentence against themselves
- By taking dot product of query matrix with key matrix of respective words



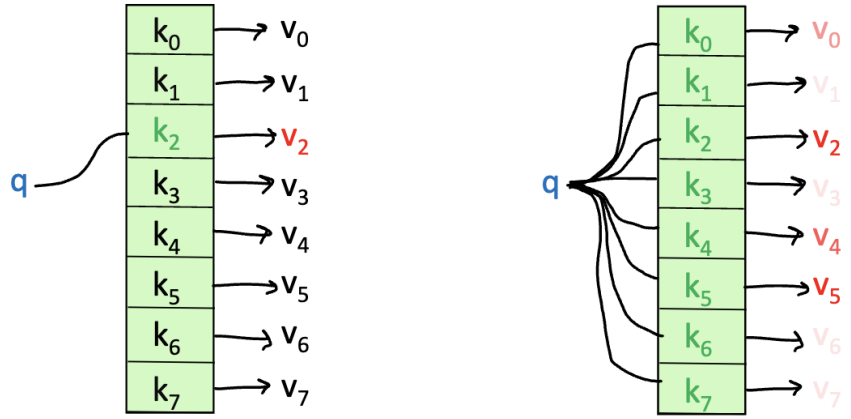
Filing Systems Analogy of Q, K, V

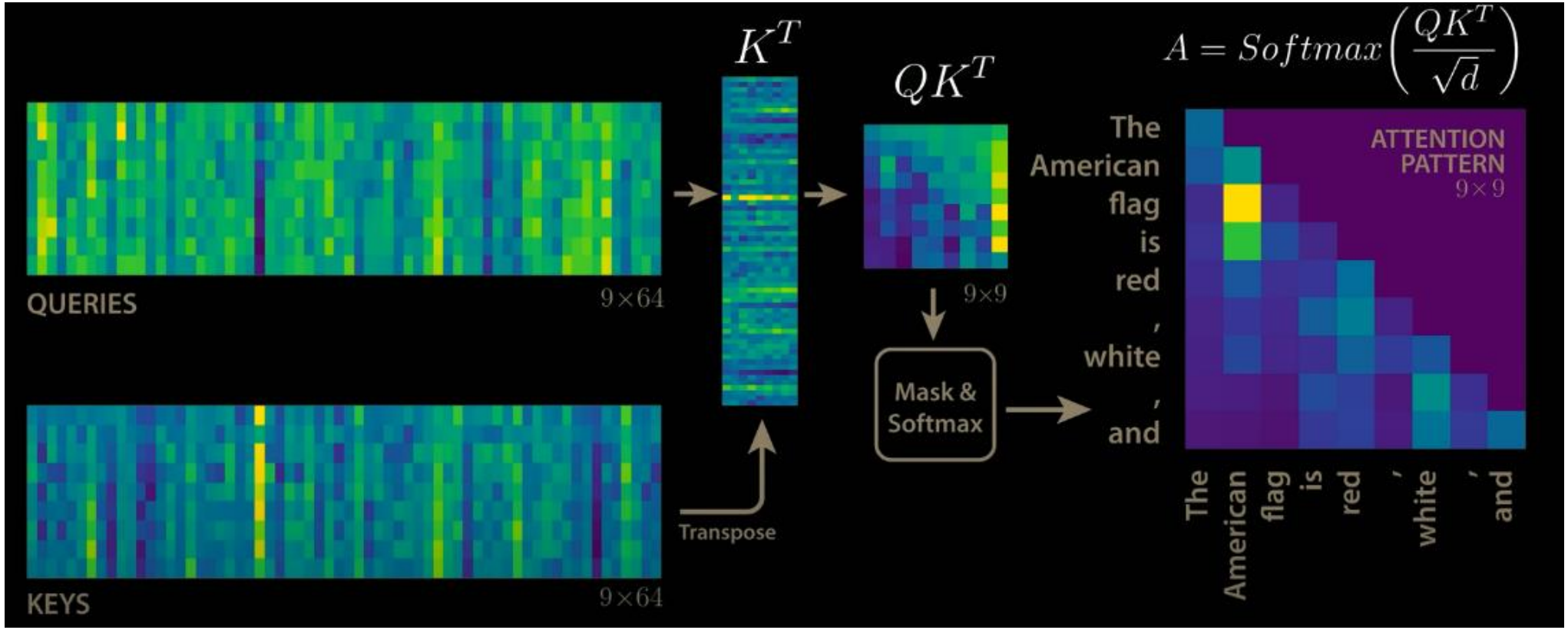
- Imagine you're looking for info on a specific topic (query)
- Each folder has a label (key) that describes the contents
- You do a soft match between the topic (query) and the book title (key) to determine which folders are most relevant.
- Once you find a match between your query and a summary, you access the folder content (value) based on relevance.



Intuition for Attention mechanism – Approximate Hashtable

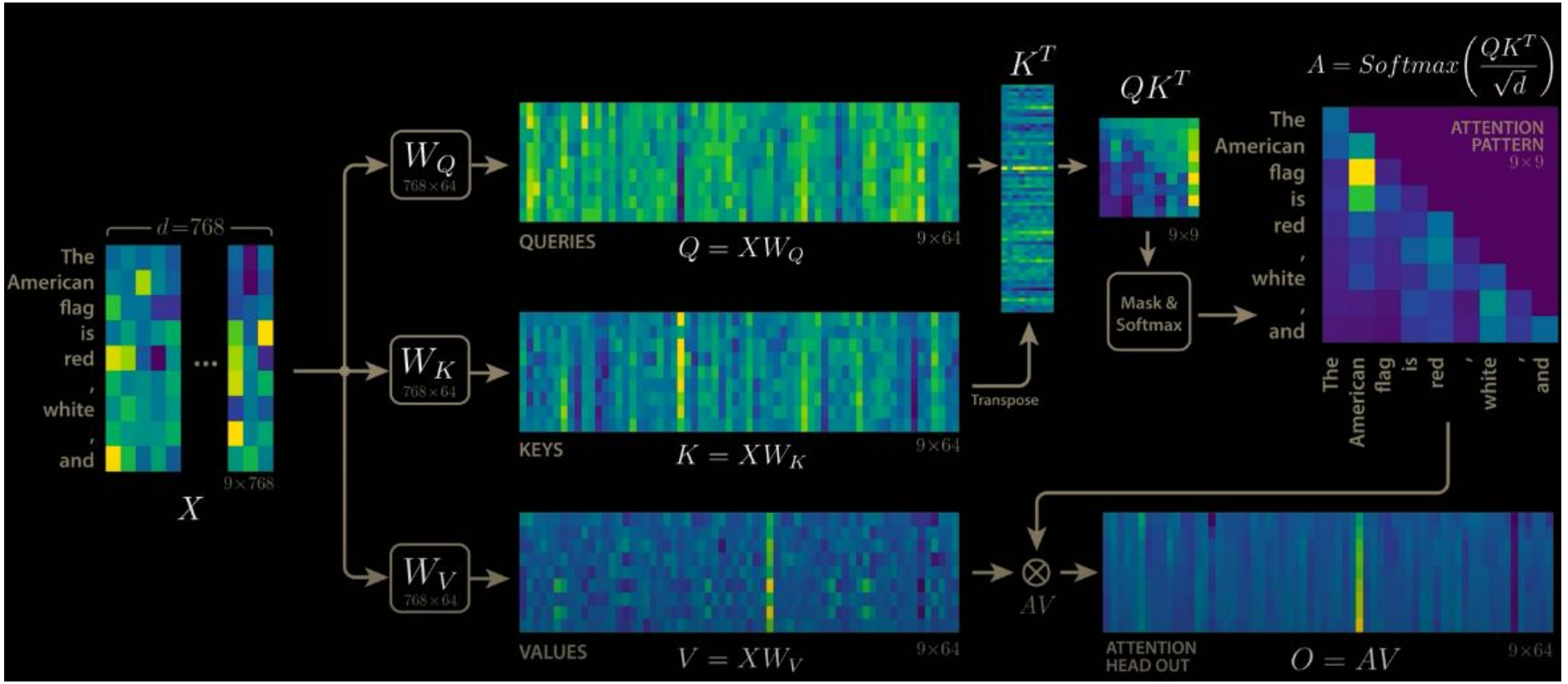
- To look up a value, we compare a query against keys in a table.
- Each query (hash) maps to exactly one key-value pair in a hash-table (left)
- Each query matches each key to varying degrees in self-attention (right)
    - We return a sum of values weighted by the query-key match.





Scaled Dot-Product Self-Attention

- Step 3: Scores then divided by sqrt(d_k)
  - Leads to more stable gradients
- Step 4: Use Softmax to get normalized probability scores; determines how much each token will be expressed at this position
- We mask future tokens to prevent model from seeing future tokens

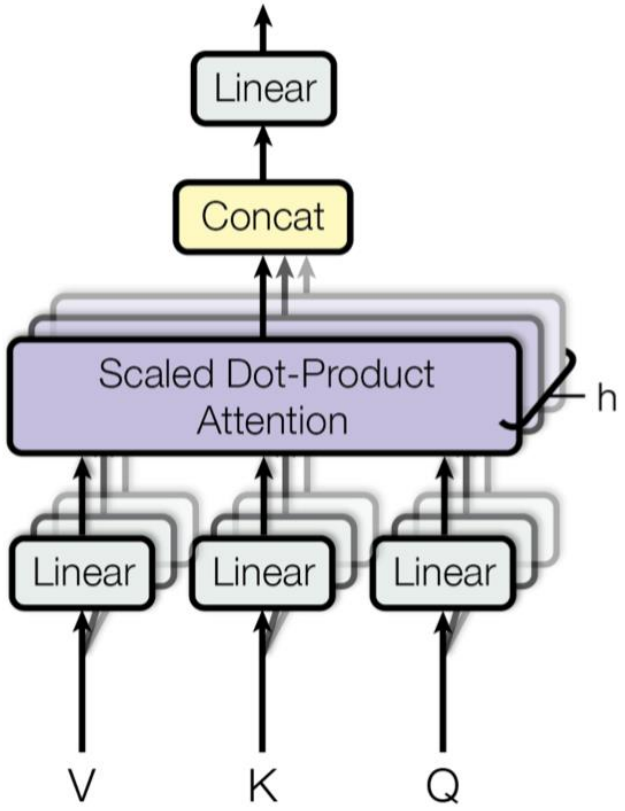


Scaled Dot-Product Self-Attention

Step 5: Multiply the value matrix with the Softmax score

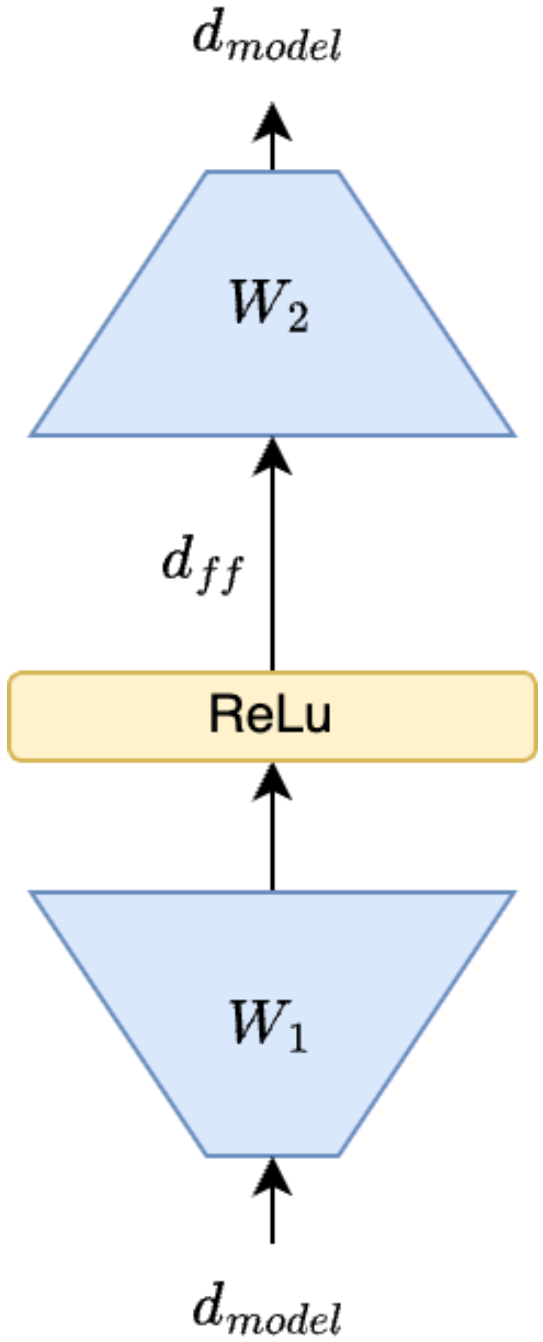
Multi-Headed Attention

- High-Level Idea: Let's perform self-attention multiple times in parallel and combine the results.
- Improves performance of the attention layer in two ways:
  - Expands model's ability to focus on different positions.
  - Each head gets to "look" at different things, and construct value vectors differently.



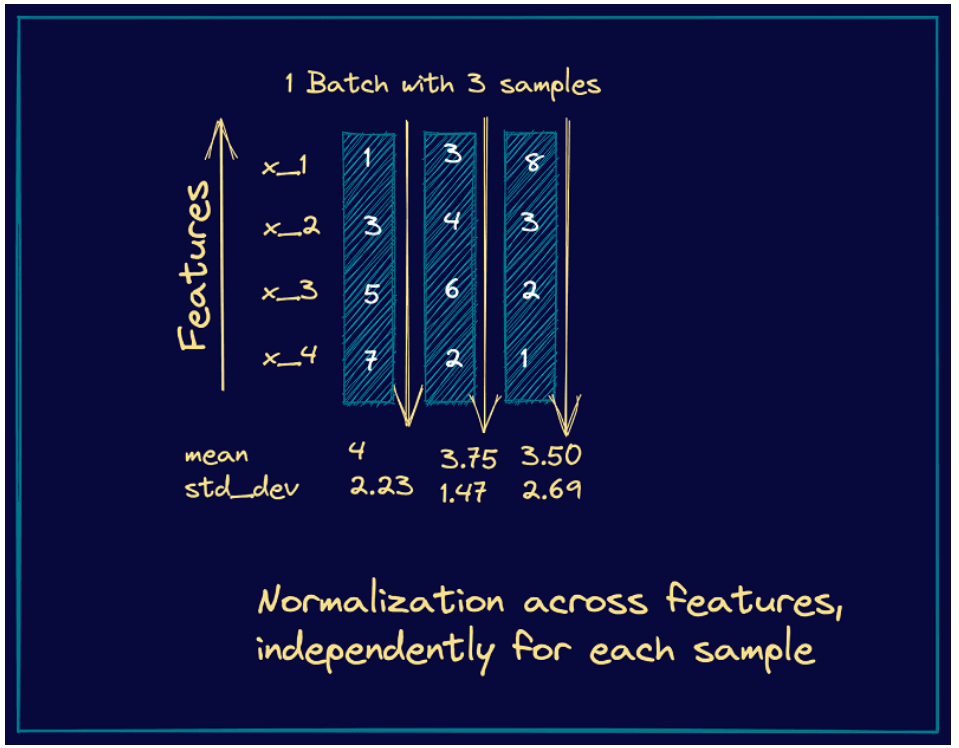
Feed-Forward Network

- Problem: Since there are no element-wise non-linearities, self-attention is simply performing a re-averaging of the value vectors.
- Easy fix: Apply a feedforward layer to the output of attention, providing non-linear activation (and additional expressive power).



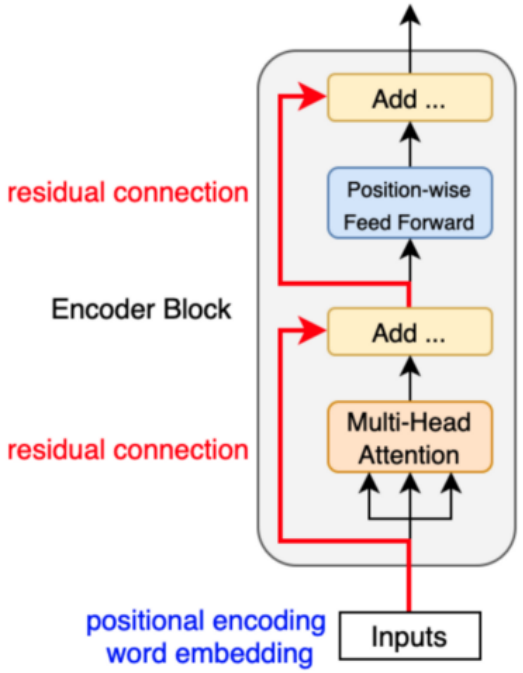
Layer Normalization

- Problem: Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting
- Solution: Reduce variation by normalizing to zero mean and standard deviation of one within each layer
- BatchNorm is hard to parallelize due to mean and variance batch statistics and performs poorly with small batch sizes



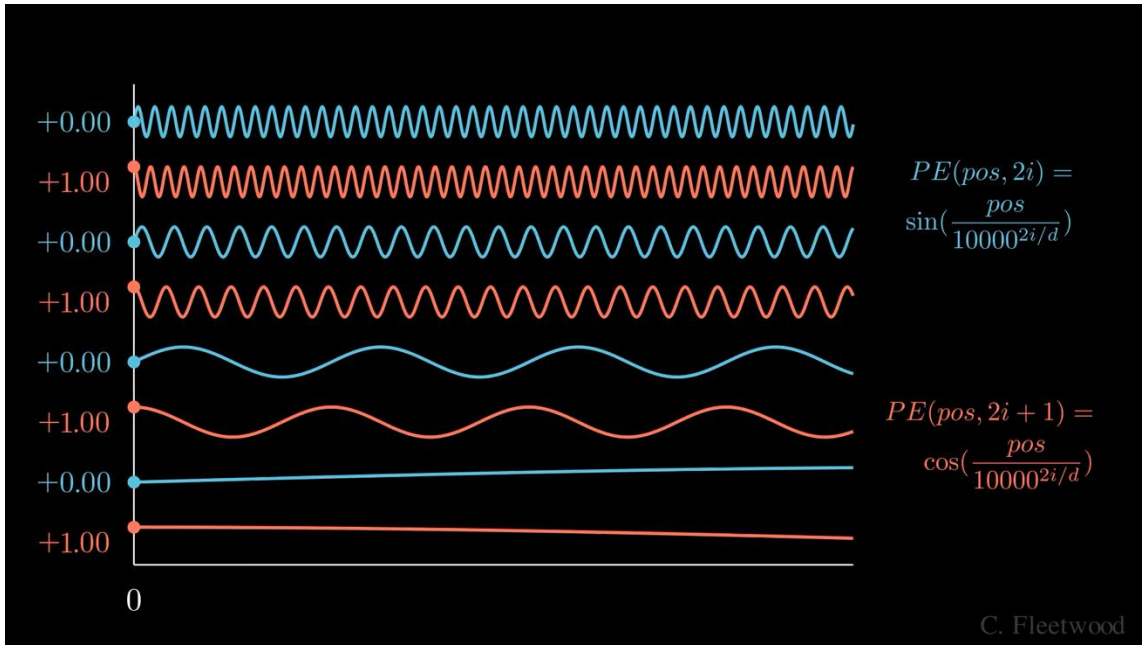
Residual Connections

- Passing "raw" embeddings to the next layer can prevent the network from "forgetting" or distorting important information as it is processed by many layers
- DNNs struggle to learn the identity function.



Sinusoidal Positional Encoding

- Self attention is a set operation, which means it is permutation equivariant
  - doesn't build in order information
- Sequence order conveys important semantic information
- Solution: Add positional information of input token in the sequence into input embedding vectors

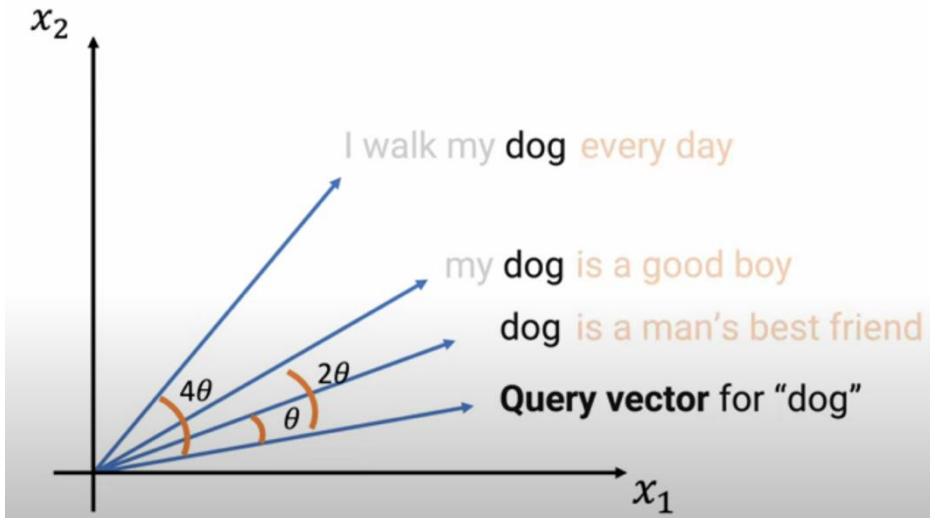


Rotary Positional Encodings (RoPE)

- Sinusoidal encodings do not capture the positional information of token relative to other tokens in a sequence
- Rotary Positional encodings include relative positional information by rotating a token's embedding by its position
- Improves performance by preventing additive pollution of embeddings

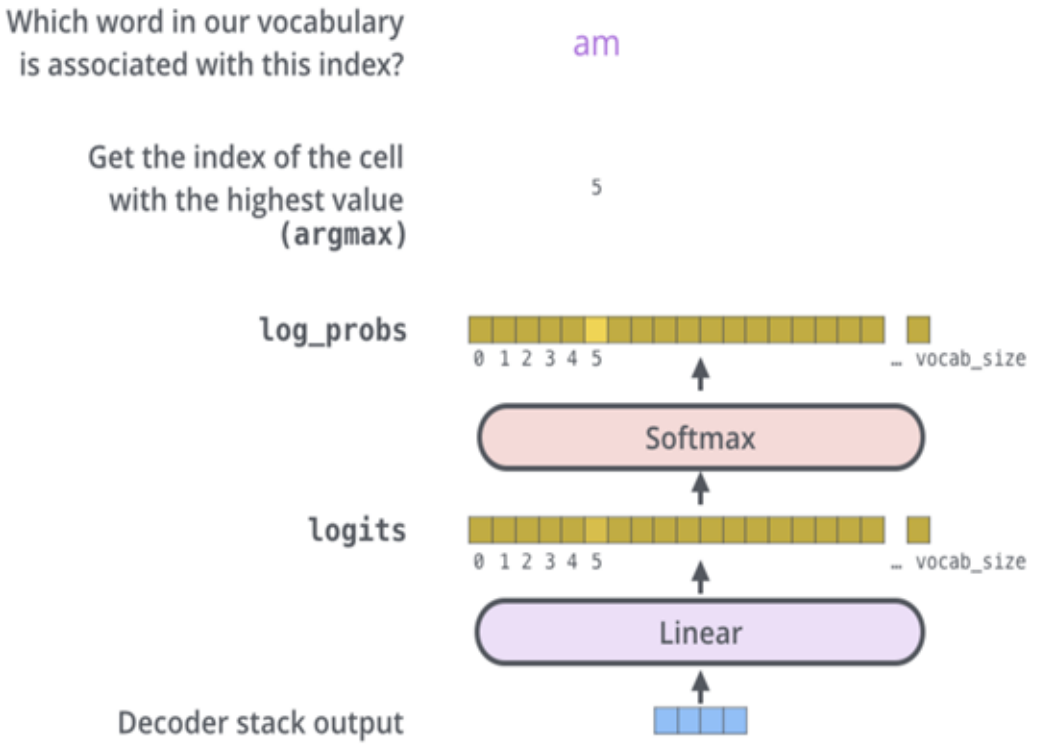
$$R^d_{\Theta,p}q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ \vdots \\ q_{d-1} \\ q_d \end{pmatrix} \otimes \begin{pmatrix} \cos p\theta_1 \\ \cos p\theta_1 \\ \cos p\theta_2 \\ \cos p\theta_2 \\ \vdots \\ \cos p\theta_{d/2} \\ \cos p\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -q_2 \\ q_1 \\ -q_4 \\ q_3 \\ \vdots \\ -q_d \\ q_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin p\theta_1 \\ \sin p\theta_1 \\ \sin p\theta_2 \\ \sin p\theta_2 \\ \vdots \\ \sin p\theta_{d/2} \\ \sin p\theta_{d/2} \end{pmatrix}$$

Position	1	2	3	4	5	6
	I	walk	my	dog	every	day
Position	1	2	3	4	5	6
	every	day	I	walk	my	dog

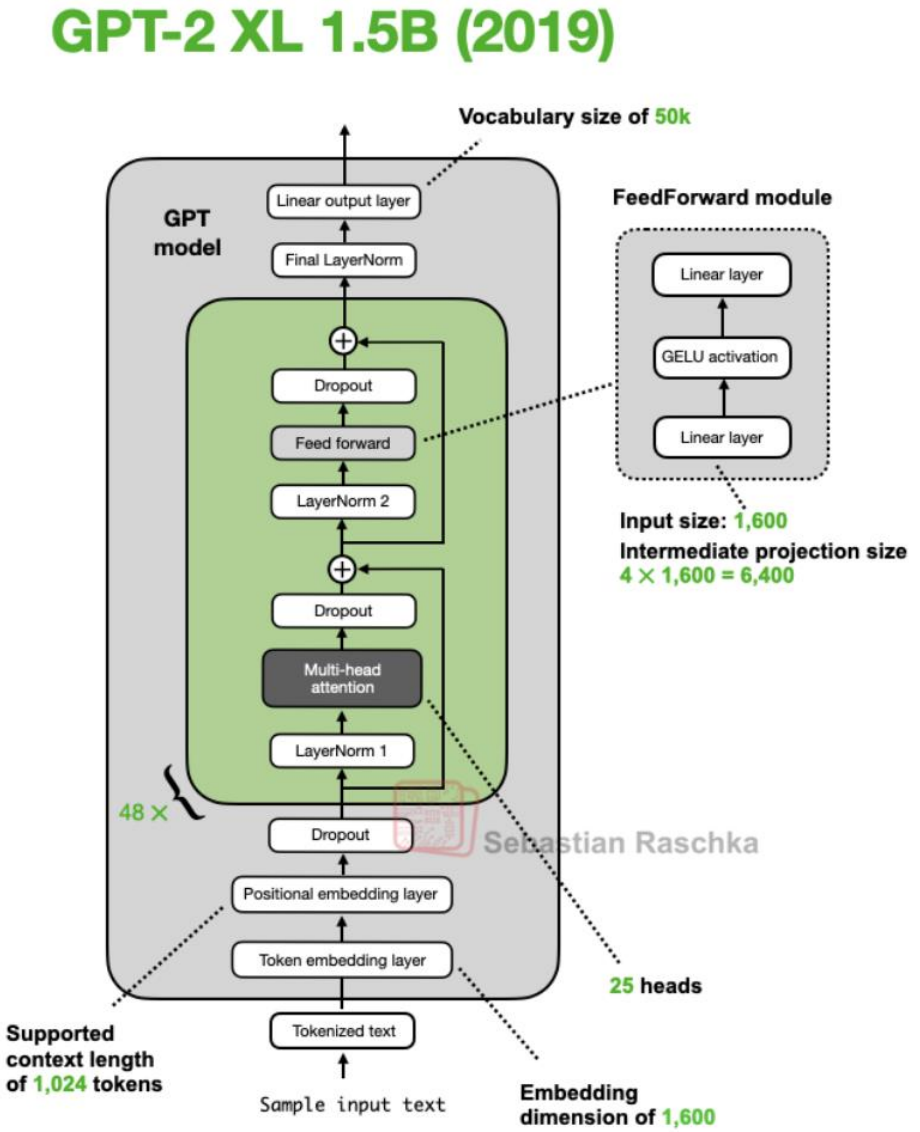


Finishing Touches

- Add a final linear layer to project the embeddings into a much longer vector of length vocab size (logits)
- Add a final Softmax to generate a probability distribution of possible next words!

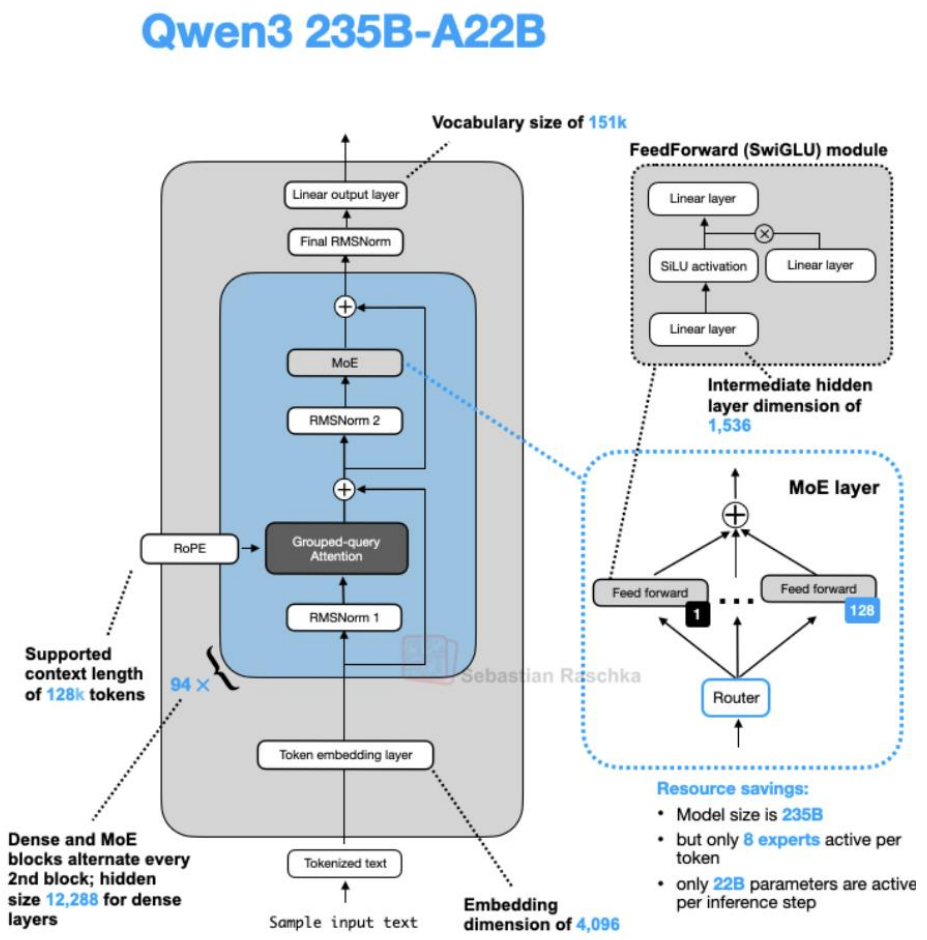


GPT-2 Architecture



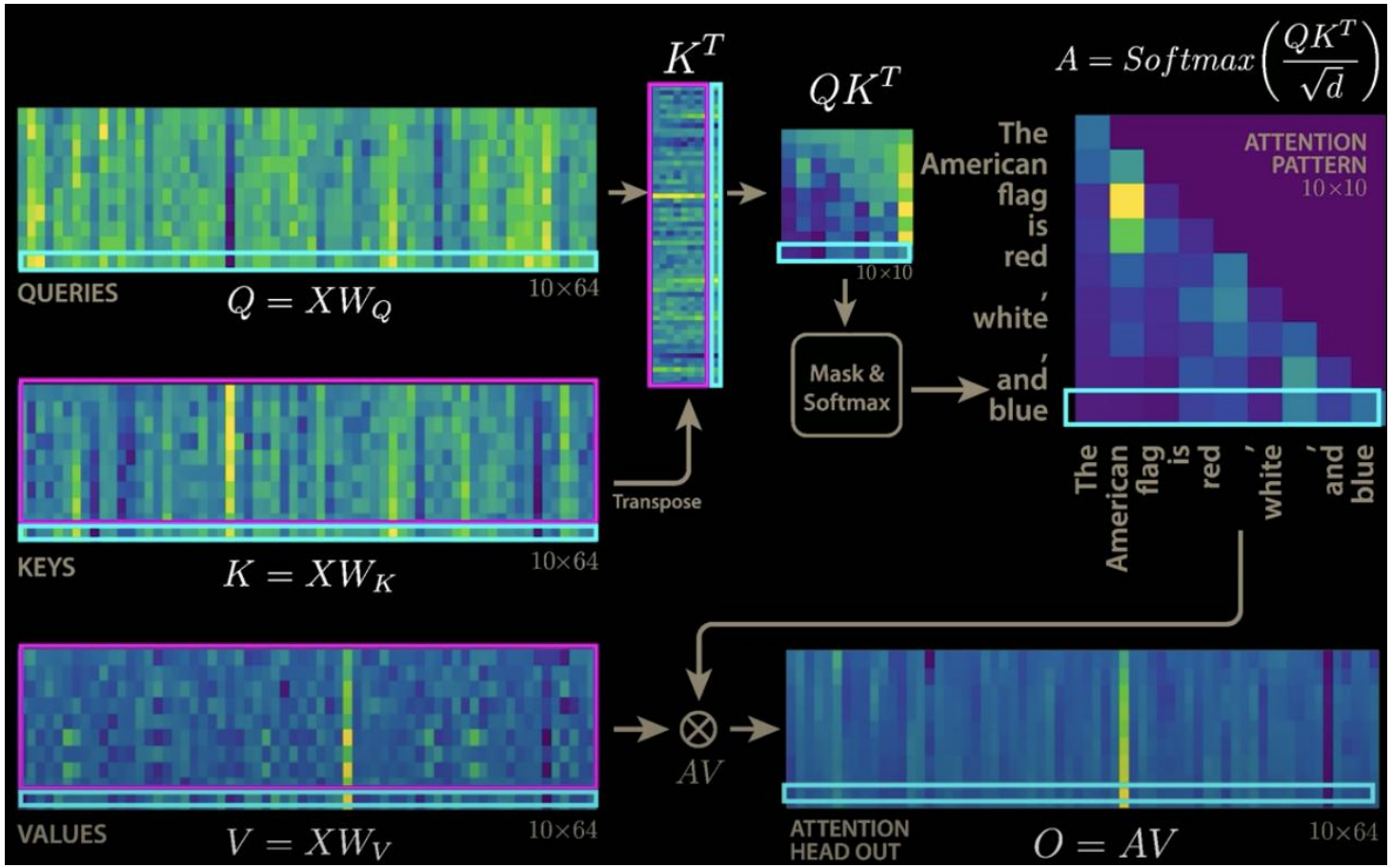
Modern LLM Architecture

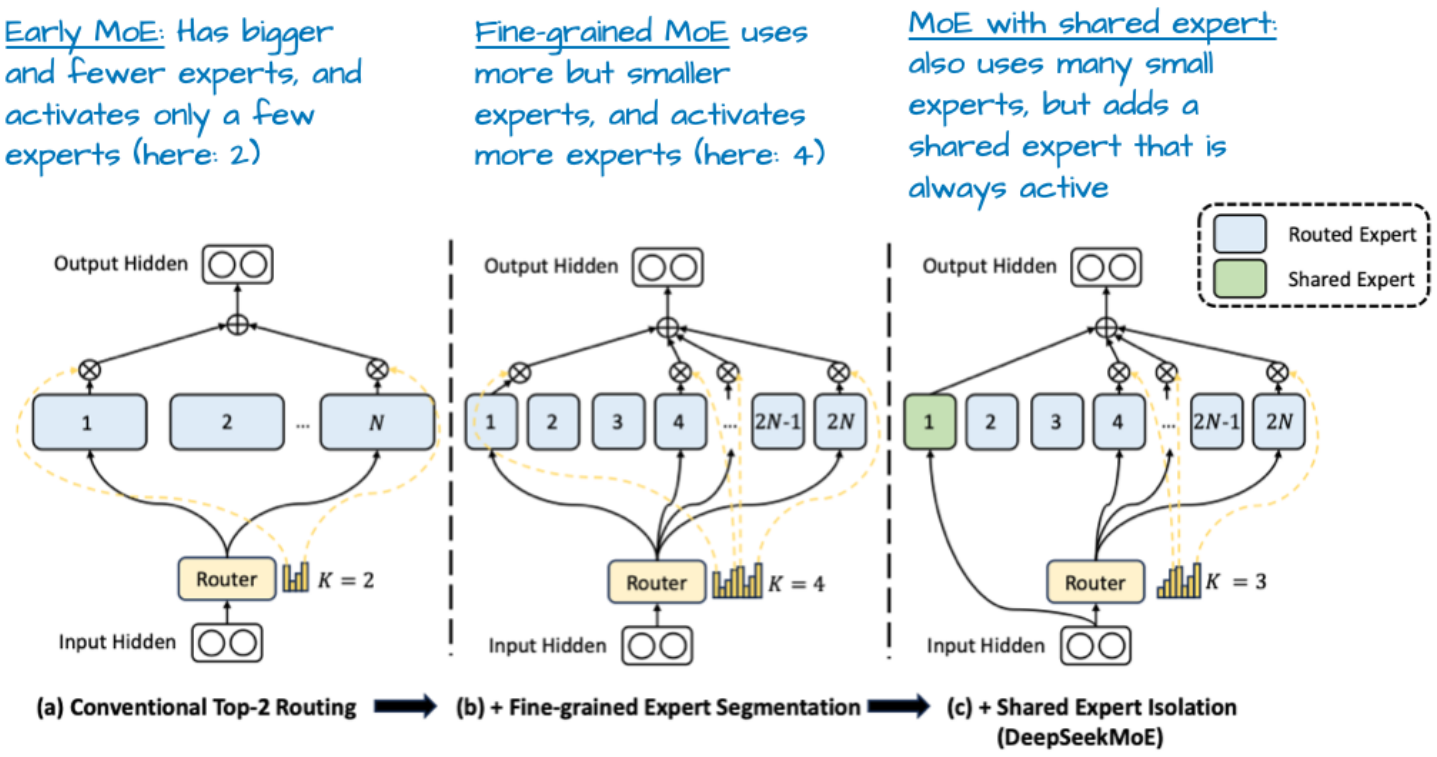
- Original Architecture is very robust
- Qwen3
  - No Dropout
  - Sinusoidal PE -> RoPE
  - GELU -> SwiGLU
  - MHA -> GQA
  - LayerNorm -> Pre-RMSNorm
  - QK-Norm
  - **KV Cache**
  - **MoE**



KV-Cache

- Store KV values of previously processed tokens in memory
- When calculating the attention scores for a new token, we retrieve old KV matrices and concatenate to the new k and v vector.
- Results in faster and more efficient text generation at the cost of increased memory usage





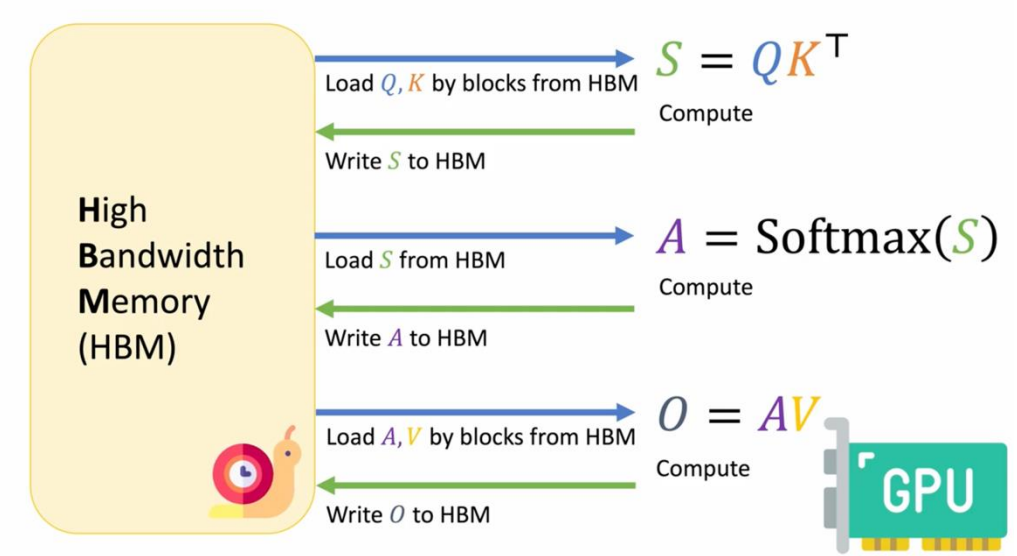
Mixture-of-Experts (shared expert)

- Deepseek V3 replaces a single FFN with multiple FFNs (experts), and has a router activate only a small subset of experts for each token.
- the large # of params increases the learning capacity of the LLM, while the sparsity keeps inference efficient
- Shared expert capture common or repeated patterns so individual experts can learn more specialized patterns.

FlashAttention

Dot-Product Attention is Inefficient

- Attention mechanism is slow and memory-hungry
- Scales quadratically with sequence length
- Bottleneck is in the memory transfers, not FLOPs



Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

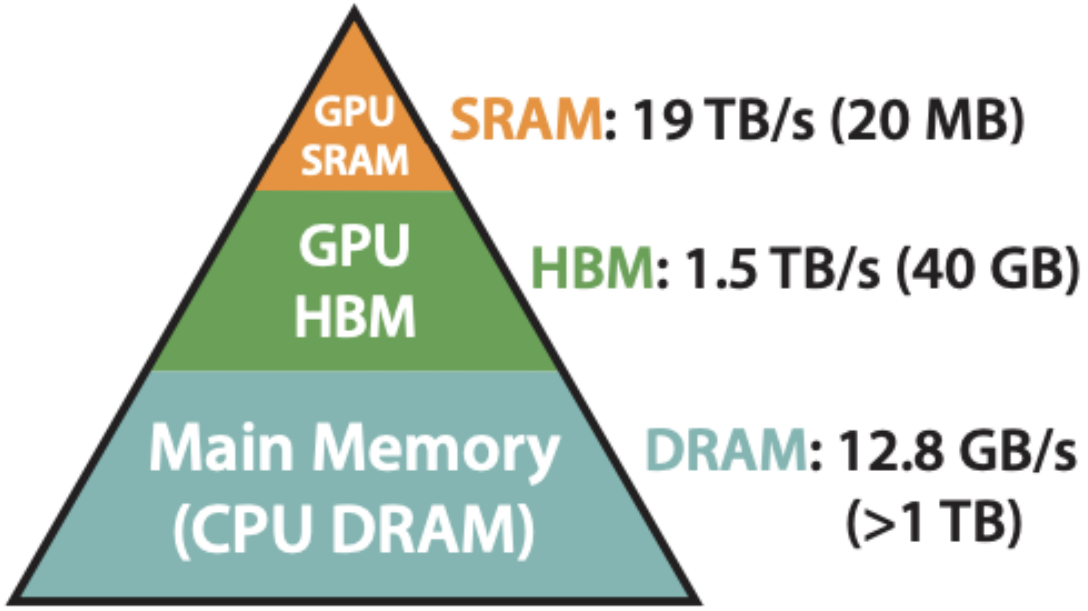
Approximate Attention Methods are Not Good Enough

- Low-rank or sparse attention methods reduce FLOPs
- Problems
  - Perf degradation
  - Hard to translate to wall-clock speedup since most ops of attention are memory bound, not compute-bound

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

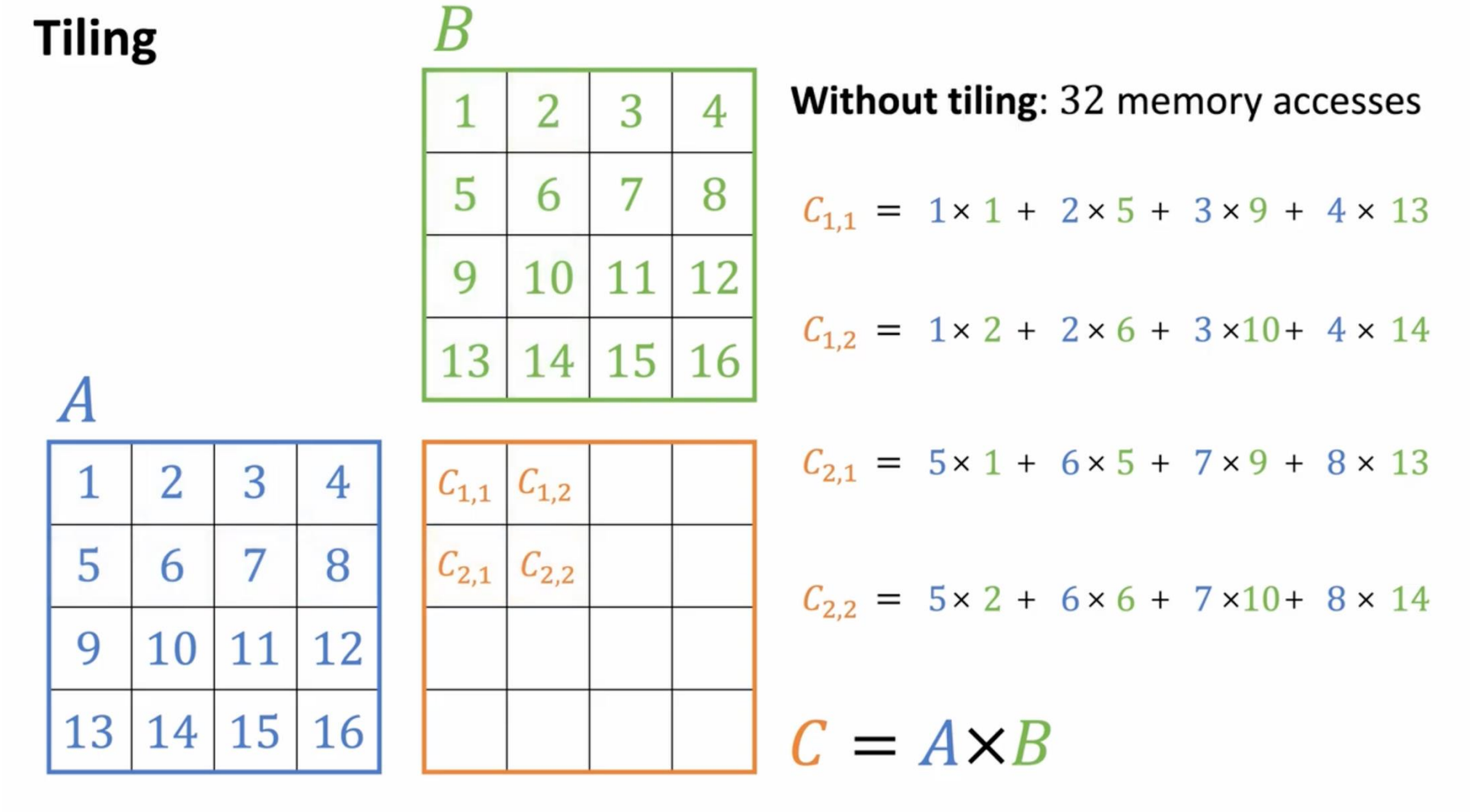
Goals of FlashAttention

- We want an exact attention algorithm
- Perform all of the computation in fast but small SRAM
- We want to reduce memory reads and writes where we can

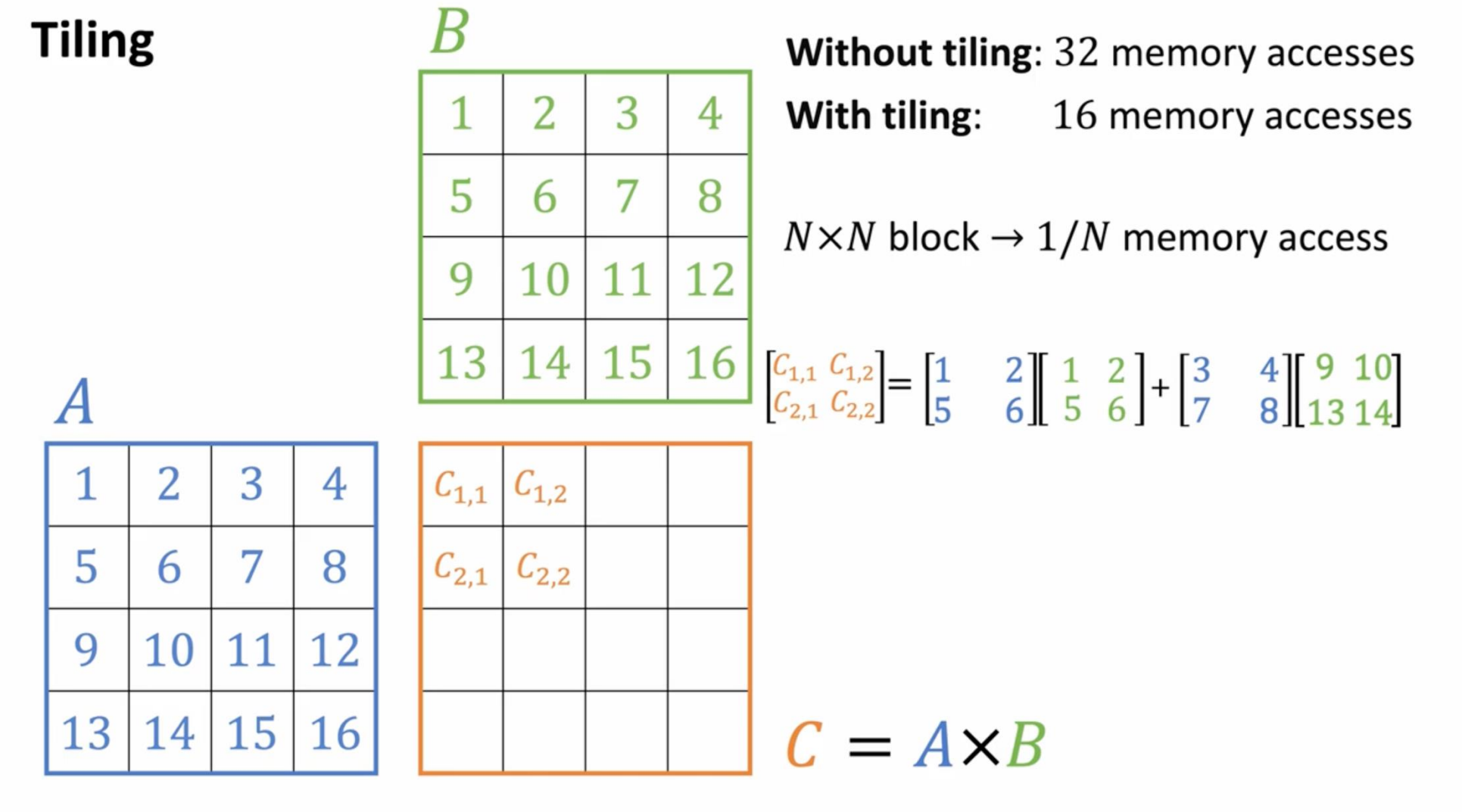


Memory Hierarchy with Bandwidth & Memory Size

Inspiration: Tiled Matrix Multiplication



Inspiration: Tiled Matrix Multiplication



Side Note: Safe Softmax

- Regular Softmax is not numerically stable
- With half precision, we get overflows when input values > 11
- Simple Solution: Subtract the row-wise max from each input in the exponential
- All input values in the exponential are now <= 0

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_k \exp(x_k)} \times \frac{\exp(-m)}{\exp(-m)}$$

$$\Rightarrow \quad \frac{\exp(x_i - m)}{\sum_k \exp(x_k - m)}$$

Softmax is problematic for tiling

- Tiling only gives us access to elements within tile/block
- Softmax requires a row-wise sum
- Safe Softmax requires a row-wise max

$$m_N = \max(\{x_1, x_2, \cdots, x_N\})$$

$$d_N = \sum_{i=1}^{N} e^{x_i - m_N}$$

$$A = \{a_1, a_2, \cdots, a_N\}$$
$$a_i = e^{x_i - m_N} / d_N$$

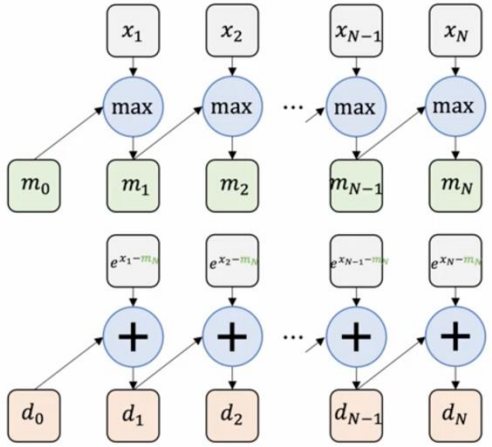
$$A = \text{Softmax}(S)$$

$$m_0 = -\infty$$
for $1 \le i \le N$ **do**
$$m_i = \max(m_{i-1}, x_i)$$

$$d_0 = 0$$
for $1 \le i \le N$ **do**
$$d_i = d_{i-1} + e^{x_i - m_N}$$

for $1 \le i \le N$ **do**
$$a_i = e^{x_i - m_N} / d_N$$

$$S = \{x_1, x_2, \cdots, x_N\}$$



SAFE SOFTMAX

Solution: Online Softmax

- We can remove d's dependency for the row-wise max
- Key Idea: When we find a new max in x, we correct the running sum by multiplying by e^{old_max - new_max}
- This way, we fuse the first two loops into one

$$A = \text{Softmax}(S) \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$m_0 = -\infty$
for $1 \le i \le N$ **do**
  $m_i = \max(m_{i-1}, x_i)$

$d_0 = 0$
for $1 \le i \le N$ **do**
  $d_i = d_{i-1} + e^{x_i - m_N}$

for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d_N$

$$d_i = \sum_{j=1}^{i} e^{x_j - m_N} \qquad d'_i = \sum_{j=1}^{i} e^{x_j - m_i} \qquad d'_N = d_N$$

$$d'_i = \sum_{j=1}^{i} e^{x_j - m_i}$$

Solution: Online Softmax

- We can remove d's dependency for the row-wise max
- Key Idea: When we find a new max in x, we correct the running sum by multiplying by e^{old_max - new_max}
- This way, we fuse the first two loops into one

$$A = \text{Softmax}(S) \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$m_0 = -\infty$
for $1 \le i \le N$ **do**
  $m_i = \max(m_{i-1}, x_i)$

$d_0 = 0$
for $1 \le i \le N$ **do**
  $d_i = d_{i-1} + e^{x_i - m_N}$

for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d_N$

$$d_i = \sum_{j=1}^{i} e^{x_j - m_N} \qquad d'_i = \sum_{j=1}^{i} e^{x_j - m_i} \qquad d'_N = d_N$$

$$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_i}\right) e^{m_{i-1}} e^{-m_{i-1}} + e^{x_i - m_i}$$

Solution: Online Softmax

- We can remove d's dependency for the row-wise max
- Key Idea: When we find a new max in x, we correct the running sum by multiplying by e^{old_max - new_max}
- This way, we fuse the first two loops into one

$$A = \text{Softmax}(S) \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$m_0 = -\infty$
for $1 \le i \le N$ **do**
  $m_i = \max(m_{i-1}, x_i)$

$d_0 = 0$
for $1 \le i \le N$ **do**
  $d_i = d_{i-1} + e^{x_i - m_N}$

for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d_N$

$$d_i = \sum_{j=1}^{i} e^{x_j - m_N} \qquad d'_i = \sum_{j=1}^{i} e^{x_j - m_i} \qquad d'_N = d_N$$

$$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_{i-1}}\right) e^{m_{i-1}} e^{-m_i} + e^{x_i - m_i}$$

Solution: Online Softmax

- We can remove d's dependency for the row-wise max
- Key Idea: When we find a new max in x, we correct the running sum by multiplying by e^{old_max - new_max}
- This way, we fuse the first two loops into one

$$A = \text{Softmax}(S) \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$m_0 = -\infty$
for $1 \le i \le N$ **do**
  $m_i = \max(m_{i-1}, x_i)$

$d_0 = 0$
for $1 \le i \le N$ **do**
  $d_i = d_{i-1} + e^{x_i - m_N}$

for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d_N$

$$d_i = \sum_{j=1}^{i} e^{x_j - m_N} \qquad d'_i = \sum_{j=1}^{i} e^{x_j - m_i} \qquad d'_N = d_N$$

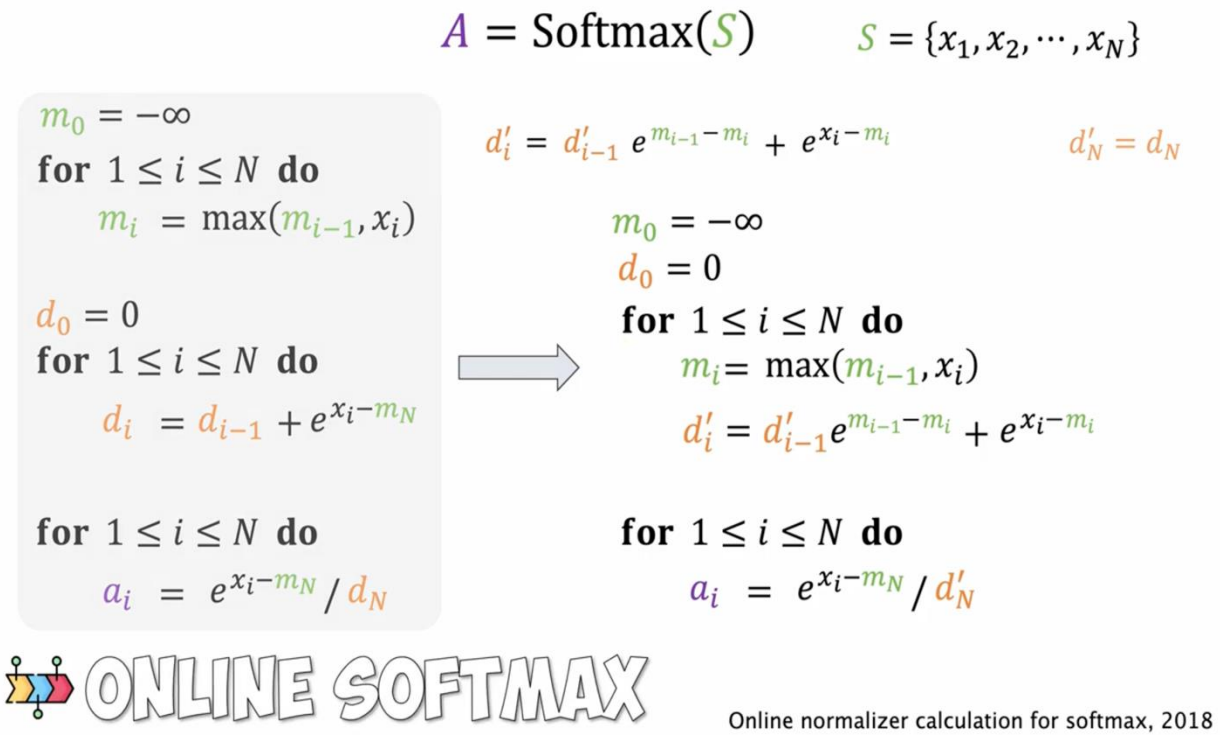
$$d'_i = \Big(\sum_{j=1}^{i-1} e^{x_j - m_{i-1}}\Big) e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

($\sum_{j=1}^{i-1} e^{x_j - m_{i-1}}$ is d'_{i-1})

SAFE SOFTMAX

Solution: Online Softmax

- We can remove d's dependency for the row-wise max
- Key Idea: When we find a new max in x, we correct the running sum by multiplying by e^{old_max - new_max}
- This way, we fuse the first two loops into one



FlashAttention: Putting it all together

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV$$

$$S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1}\, e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$m_0 = -\infty$
$d_0 = 0$
for $1 \le i \le N$ **do**
  $x_i = qk_i^\top$
  $m_i = \max(m_{i-1}, x_i)$
  $d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$
$o_0 = 0$
for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d'_N$
  $o_i = o_{i-1} + a_i v_i$

FlashAttention: Putting it all together

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1}\, e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o_i = \sum_{j=1}^{i} \frac{e^{x_j - m_N}}{d'_N} v_j \qquad o_N = o'_N$$

$$o'_i = \sum_{j=1}^{i} \frac{e^{x_j - m_i}}{d'_i} v_j$$

$m_0 = -\infty$
$d_0 = 0$
for $1 \le i \le N$ **do**
  $x_i = qk_i^\top$
  $m_i = \max(m_{i-1}, x_i)$
  $d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$
$o_0 = 0$
for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d'_N$
  $o_i = o_{i-1} + a_i v_i$
return o_N

FlashAttention: Putting it all together

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1}\, e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = \sum_{j=1}^{i-1} \frac{e^{x_j - m_i}}{d'_i} \frac{d'_{i-1}}{d'_{i-1}} \frac{e^{-m_{i-1}}}{e^{-m_{i-1}}} v_j + \frac{e^{x_i - m_i}}{d'_i} v_i$$

$m_0 = -\infty$
$d_0 = 0$
for $1 \le i \le N$ **do**
  $x_i = qk_i^\top$
  $m_i = \max(m_{i-1}, x_i)$
  $d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$
$o_0 = 0$
for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d'_N$
  $o_i = o_{i-1} + a_i\, v_i$
return o_N

FlashAttention: Putting it all together

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1}\, e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = \sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}} d'_{i-1}}{d'_{i-1}} \frac{e^{-m_i}}{d'_i} \frac{}{e^{-m_{i-1}}} v_j + \frac{e^{x_i - m_i}}{d'_i} v_i$$

$m_0 = -\infty$
$d_0 = 0$
for $1 \le i \le N$ **do**
  $x_i = qk_i^\top$
  $m_i = \max(m_{i-1}, x_i)$
  $d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$
$o_0 = 0$
for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d'_N$
  $o_i = o_{i-1} + a_i\, v_i$
return o_N

FlashAttention: Putting it all together

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1}\, e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}}}{d'_{i-1}} v_j\right) \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

o'_{i-1}

$m_0 = -\infty$
$d_0 = 0$
for $1 \le i \le N$ **do**
  $x_i = qk_i^\top$
  $m_i = \max(m_{i-1}, x_i)$
  $d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$
$o_0 = 0$
for $1 \le i \le N$ **do**
  $a_i = e^{x_i - m_N} / d'_N$
  $o_i = o_{i-1} + a_i\, v_i$
return o_N

FlashAttention: Putting it all together

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV$$

$$S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

for $1 \le i \le N$ **do**

$$x_i = qk_i^\top$$

$$m_i = \max(m_{i-1}, x_i)$$

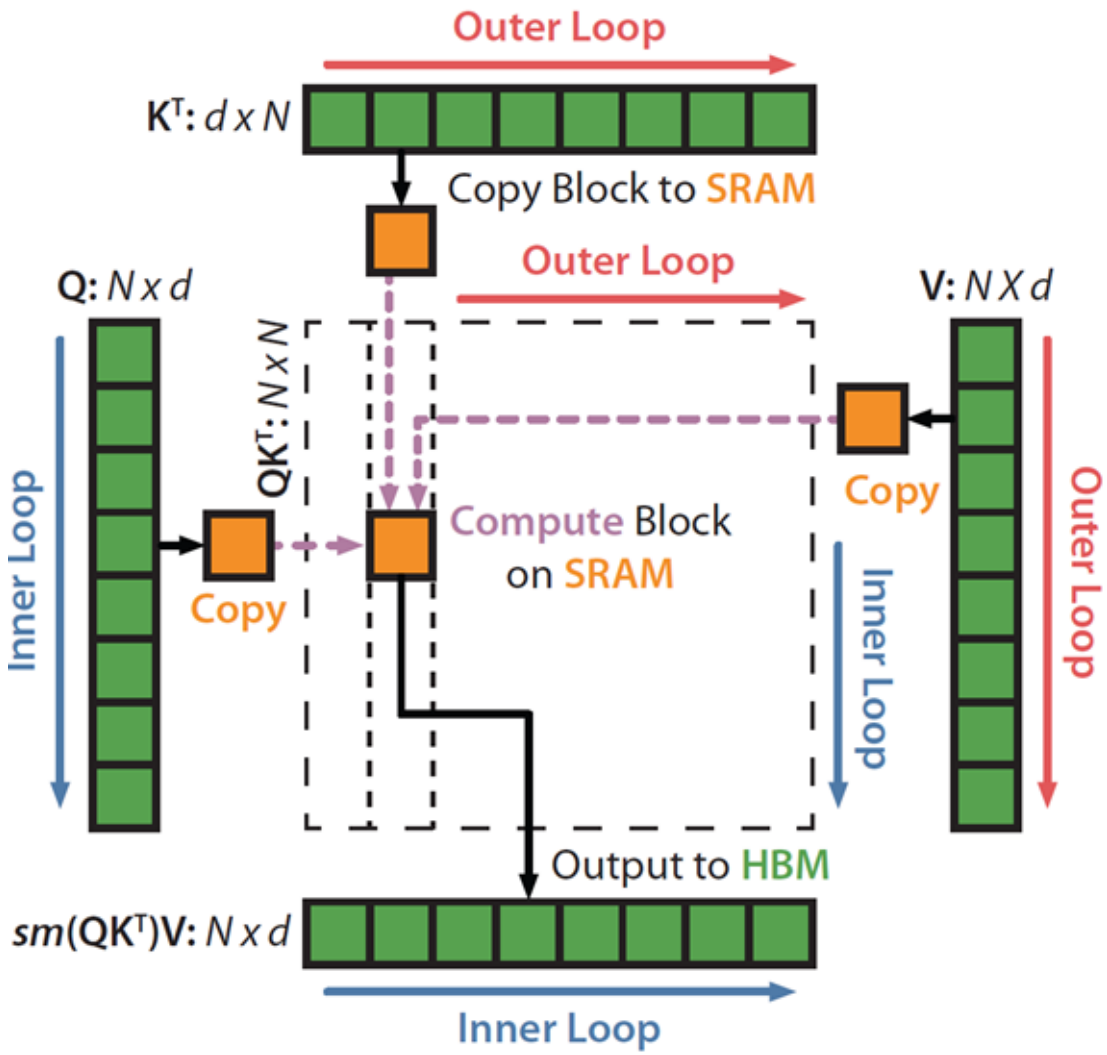
$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = o'_{i-1} \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

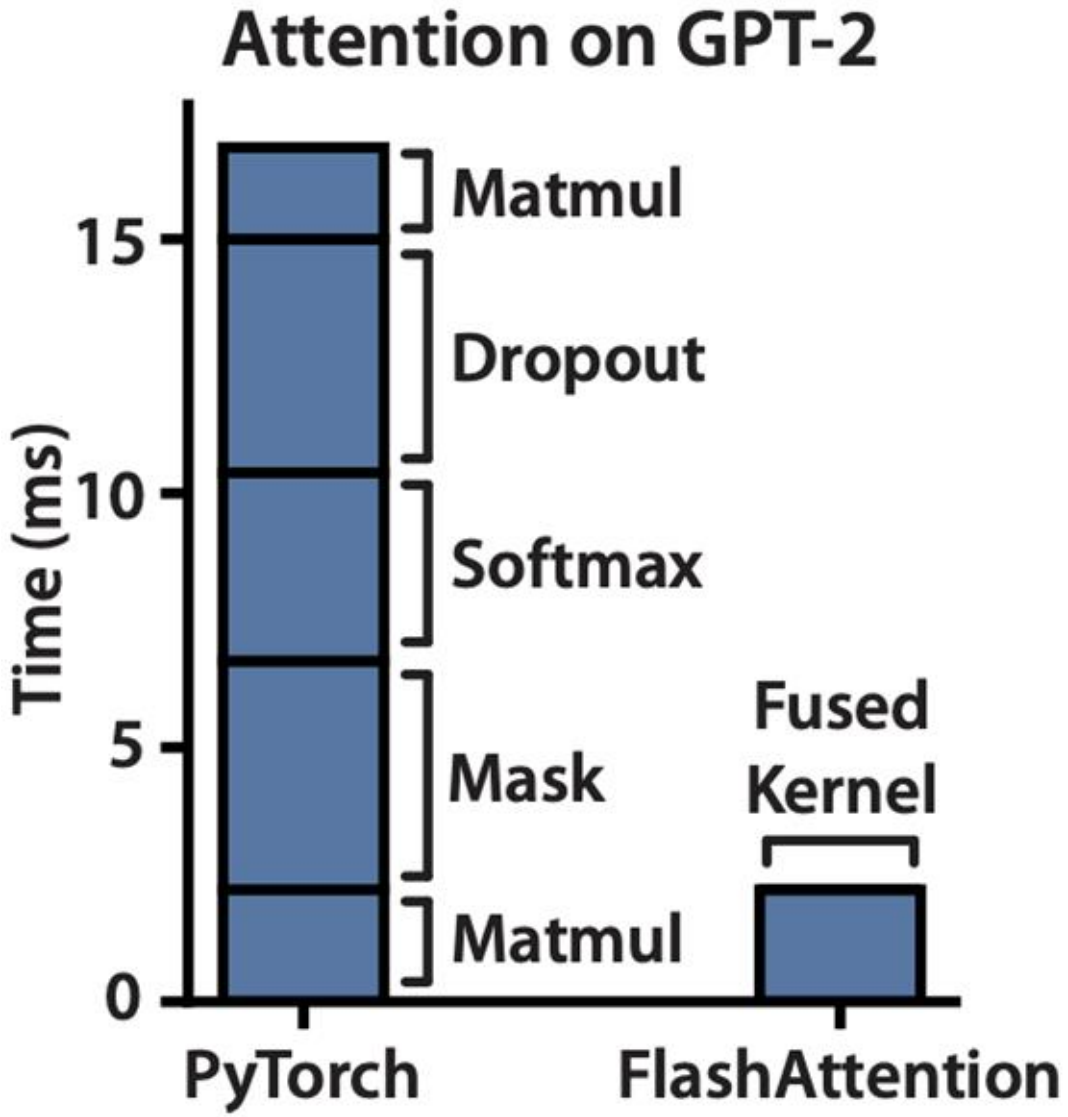
return o'_N

Fast and Memory-Efficient Exact Attention with IO-Awareness, 2022

FlashAttention: Putting it all together



Final Improvement: Kernel Fusion



Summary

- We introduced the original transformer architecture and some modern improvements to it.
- We showed how FlashAttention achieves speedups by addressing memory bottlenecks in attention